▶ this :

* "this" is a keyword.
* It is a non-static variable which can be used only inside non-static context.
* "this" keyword will always have the address of object.

purpose of 'this' keyword :

* It is used to access non-static members of the class.
* Whenever we are in a situation where global variable & local variable are having the same name, to access the global variable, we can use 'this' keyword.

Ex:

```
class M3
{
int a;
public void test()
{
int a=30;
System.out.println(a);
System.out.println(this);
System.out.println(this.a);
}
public static void main(String[] args)
{
int a=20;
System.out.println(a);
System.out.println(a)
M3 obj = new M3();
obj.test();
}
}
```

Op:

```
20
30
M3@15db9748
0
```

NOTE:

'this' keyword will not just have address of the object, it is always the ~~will have~~ address of objects which are currently in use or latest object.

Eg:
```
class M4
{
int a;
public void test()
{
int a=30;
System.out.println(a);
System.out.println(this);
System.out.println(this.a);
}
public static void main (String[] args)
{
int a=20;
System.out.println(a);
M4 obj= new M4();
obj.test();
M4 obj1= new M4();
obj1.test();
}
}
```

O/P:

```
20
30
M4@15db9748
0
30
M4@------
0
```

16/02/2022.

MEMBERS/USAGE	STATIC CONTEXT	NON-STATIC CONTEXT
STATIC MEMBERS	1) DIRECTLY ~~class N.~~ 2) class Name 3) Object Reference	1) Directly ~~2) class Name~~ 2) Object reference 3) this
NON-STATIC MEMBERS	Only with Object Reference.	1) Directly. 2) this keyword. 3) Object reference.

NON- STATIC INITIALIZER

In java we have following non-static initializers.

(i) Non-static declare & initialization statement.

(ii) Non-static block

▶ NON-STATIC DECLARE & INITIALIZATION STATEMENT:

SYNTAX:

Datatype identifier = value / Expression;

Eg:

a) int a = 10; → initialization // Global area.
↙ declaration

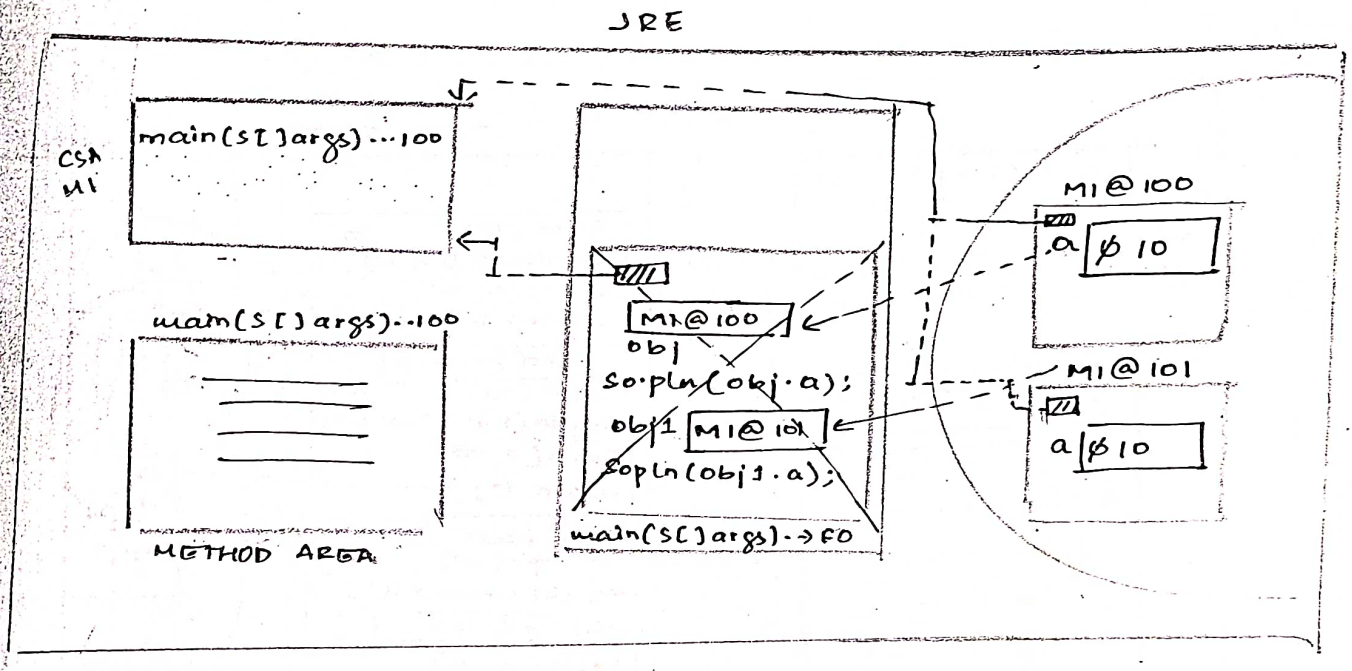
b) double sal = 1000 + 3000;
↙ declaration ↘ initialization.

c) int c = test();

Eg:
```
class M1
{
int a=10;
public static void main(String[]args)
{
M1 obj=new M1();
System.out.println(obj.a);
```

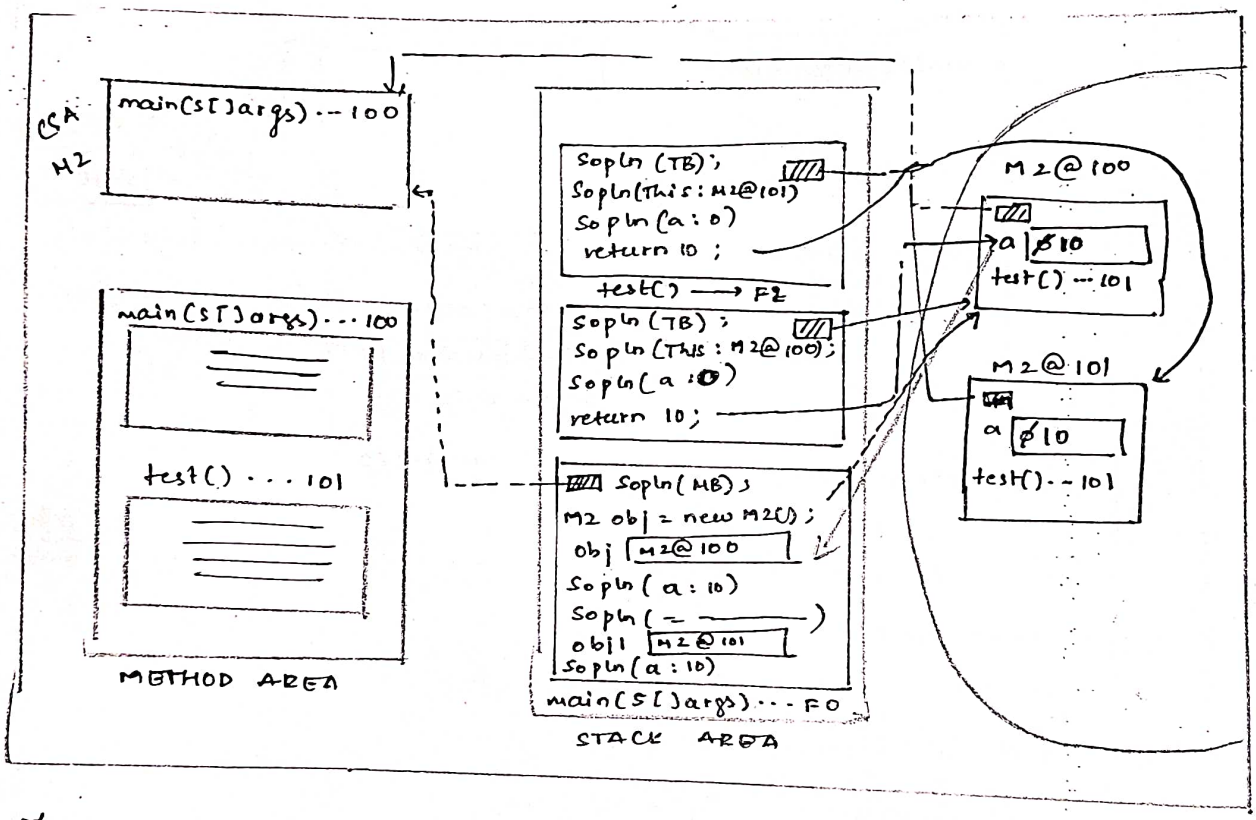
```
M1 obj1 = new M1();
System.out.println(obj1.a);
}
}
```

o/p :
10
10



Eg:
```
class M2
{
int a= test();
int test()
{
System.out.println("Test Begin");
System.out.println("This : " + this);
System.out.println("variable a :" + a);
System.out.println("Test end");
return 10;
}
public static void main(String[] args)
{
System.out.println("Main Begin");
M2 obj = new M2();
System.out.println("variable a :" obj.a);
```

```
System.out.println("==========");
M2 obj1 = new M2();
System.out.println("variable a:" + obj1.a);
System.out.println("Main end");
}
}
```



o/p:

```
Main Begin
Test begin
This : M2@100
variable a:0
Test End
variable a:10
==========
Test Begin
This : M2@101
variable a:0
Test End
variable a:10
Main end
```

An Anonymous block which is not pre

with static modifier is known as Non-static

The non-static blocks gets executed during loa

process of object ie it gets executed ~~from~~ for

each & every object created.

Eg:

```
class M4
{
 {
  System.out.println("IIB-1");
 }
 public static void main(String[] args)
 {
  System.out.println("from main");
  new M4();
  new M4();
  new M4();
 }
}
```

o/p:

```
From main
IIB-1
IIB-1
IIB-1
```

NON-STATIC BLOCK:

An Anonymous block which is not prefixe with static modifier is known as Non-static blo

The non-static blocks gets executed during loadi process of object ie it gets executed ~~from~~ for each & every object created.

Eg:

```
class M4
{
  {
    System.out.println("IIB-1");
  }
  public static void main(String[] args)
  {
    System.out.println("from main");
    new M4();
    new M4();
    new M4();
  }
}
```

O/P:

```
From main
IIB-1
IIB-1
IIB-1
```

NOTE: If we have more than one non-static block inside the class, they get executed in top to bottom order.

Eg:
```
class M5
{
  {
   System.out.println("IIB-1");
  }
  public static void main(String[] args)
  {
   System.out.println("From main");
   new M5();
   new M5();
  }
  {
   System.out.println("IIB-2");
  }
}
```

O/P:

From main

IIB-1

IIB-2

IIB-1

IIB-2

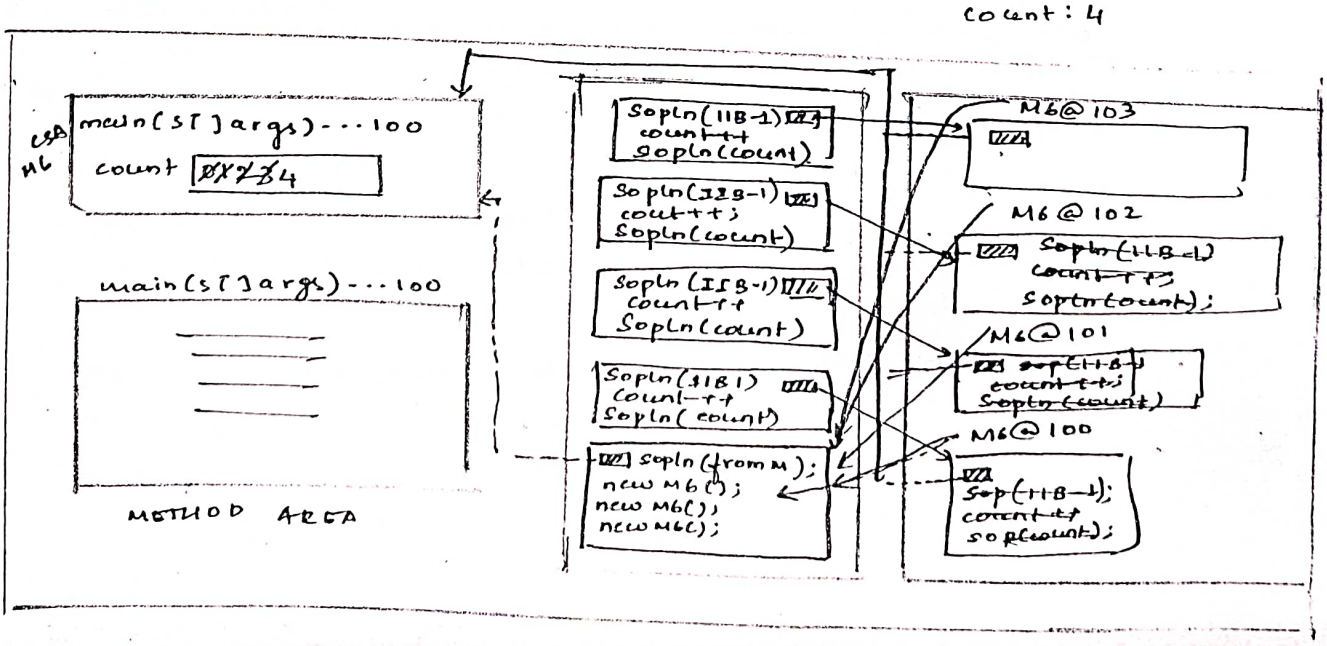
PURPOSE OF NON-STATIC BLOCK:

i) The non-static block is used to count the number of objects created.

Eg:

```
class M6
{
  static int count;
  {
    System.out.println("IIB-1");
    count++;
    System.out.println("count : " + count);
  }
  public static void main(String [] args)
  {
    System.out.println("from main");
    new M6();
    new M6();
    new M6();
    new M6();
  }
}
```

o/p: from main
IIB-1
count: 1
IIB-1
count: 2
IIB-1
count: 3
IIB-1
count: 4



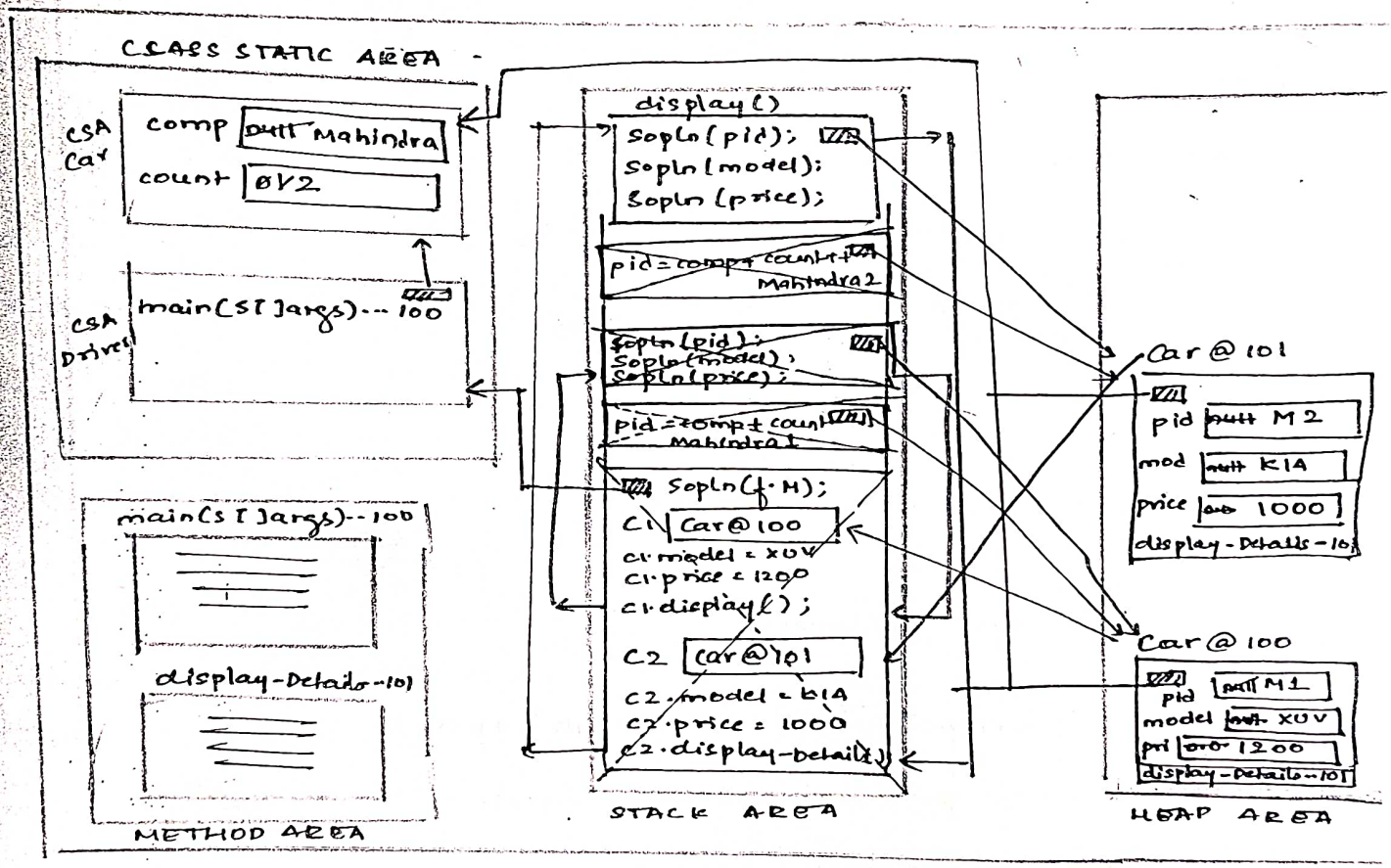
11) Usage of non-static block in the real time.

The non-static blocks are used to generate the ID's automatically to the objects.

Eg:
```
class Car
{
String pid;
String model;
double price;
static String comp = "Mahindra";
static int cocnt;
{
 pid = comp+ ++cocnt;
}
public void display_Details()
{
System.out.println("product ID: "+pid);
System.out.println("Model of the product: "+model);
System.out.println("Price of the product:"+ price);
System.out.println("==========================");
}
}

class Driver1
{
 public static void main(String[] args)
 {
  System.out.println("From main");
  Car c1 = new Car();
  c1.model = "XUV";
  c1.price = 1200;
```

```
c1. display-Details();
Car c2 = new Car();
c2. model = "KIA";
c2. price = 1000;
c2. display-Details();
}
}
```



O/P :

From main

product ID: Mahindra 1

Model of the product: XUV

Price of the product : 1200.0

= = = = = = = = = = = = = = = = = =

product ID: Mahindra 2

Model of the product: KIA

price of the product: 1000.0

= = = = = = = = = = = = = = = = = =

Assignment:

⇒ Design the blueprint for the following objects by generating their ID's automatically & display details of objects by Initializing them for a minimum of 3 objects.

(i) Laptop (ii) Mobile (iii) Marker (iv) Book (v) projector (vi) chair (vii) AC. (viii) water bottle (ix) Mic (x) Bag (xi) water (xii) Human (xiii) Lion.

LAPTOP:

```
class Laptop
{
                → static String comp = "DELL";
String pid;
                → static int count;
String model;
double price;
{
pid = comp + ++count;
}
public void display()
{
System.out.println("product ID" :+pid);
System.out.println("Model of product:" + model);
System.out.println("price of the product :" + price);
}
}
class Driver1
{
public static void main (String [] args)
{
Laptop L1 = new laptop();
L1.model = "DW07";
L1.price = 1250;
L1.display();
```

```
Laptop L2 = new Laptop();
L2.model = "DW08";
L2.price = "1500;
L2.display();
Laptop L3 = new Laptop();
L3.model = "DW10";
L3.price = 2000;
L3.display();
}
}
```